

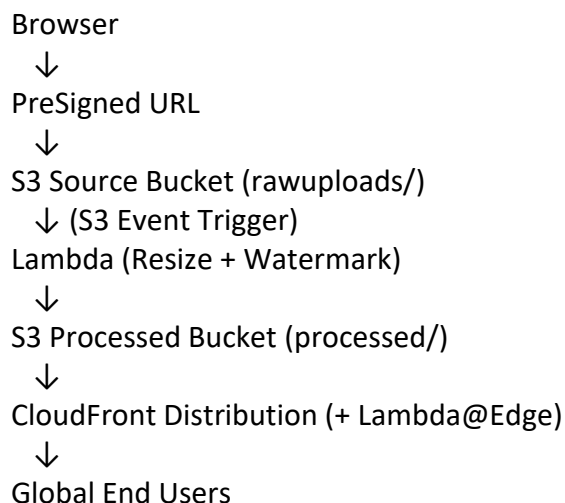
Serverless Media Processing Pipeline (AWS)

Project Overview

A fully serverless architecture designed to process large volumes of useruploaded images. The system:

- Accepts uploads via **PreSigned URLs**
 - Automatically resizes and watermarks images using **AWS Lambda**
 - Stores processed images separately
 - Distributes content globally via **Amazon CloudFront**
 - Archives original images using lifecycle policies
 - Ensures fault tolerance using **SQS DLQ**
 - Replicates data crossregion for durability
-

Architecture Flow



Serverless Media Processing Pipeline — Complete Steps

1. S3 Setup

1.1 Create Source Bucket

- 1. Go to AWS Console → S3
- 2. Click Create bucket
- 3. Name: `rawuploadsbucket`
- 4. Enable Versioning
- 5. Enable Default Encryption → SSE-S3

- 6. Create bucket

Screenshots:

The screenshot displays the AWS console 'Create bucket' page for the us-east-1 region. The page is divided into several sections, each with a red box highlighting specific settings:

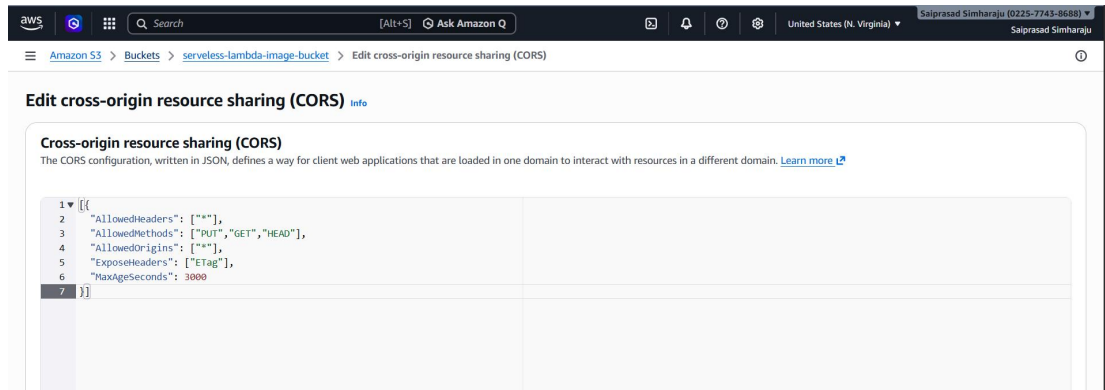
- Object Ownership**: The 'ACLs disabled (recommended)' option is selected, indicating that all objects in the bucket are owned by this account and its objects are specified using only policies.
- Block Public Access settings for this bucket**: The 'Block all public access' checkbox is checked, indicating that public access to buckets and objects granted through new access control lists (ACLs) is blocked.
- Bucket Versioning**: The 'Enable' radio button is selected, indicating that versioning is enabled for the bucket.
- Tags - optional**: The 'Add new tag' button is visible, indicating that no tags are currently associated with the bucket.
- Default encryption**: The 'Server-side encryption with Amazon S3 managed keys (SSE-S3)' option is selected, indicating that server-side encryption is automatically applied to new objects stored in this bucket.

The page also includes a 'Bucket Key' section with the 'Enable' radio button selected, and an 'Advanced settings' section at the bottom.

1.2 Configure CORS

- 1. Open `rawuploadsbucket`
- 2. Go to Permissions → CORS configuration
- 3. Add CORS rules
- 4. Save

Screenshot:



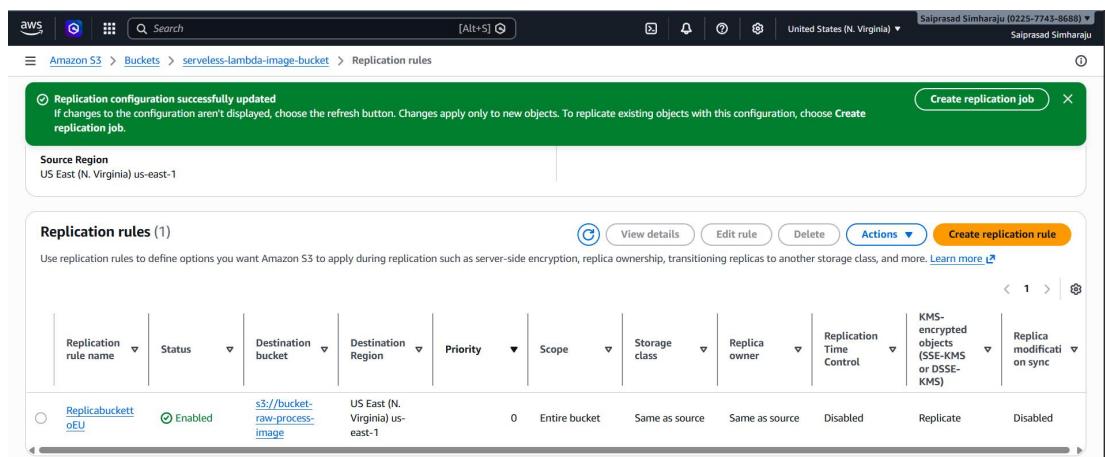
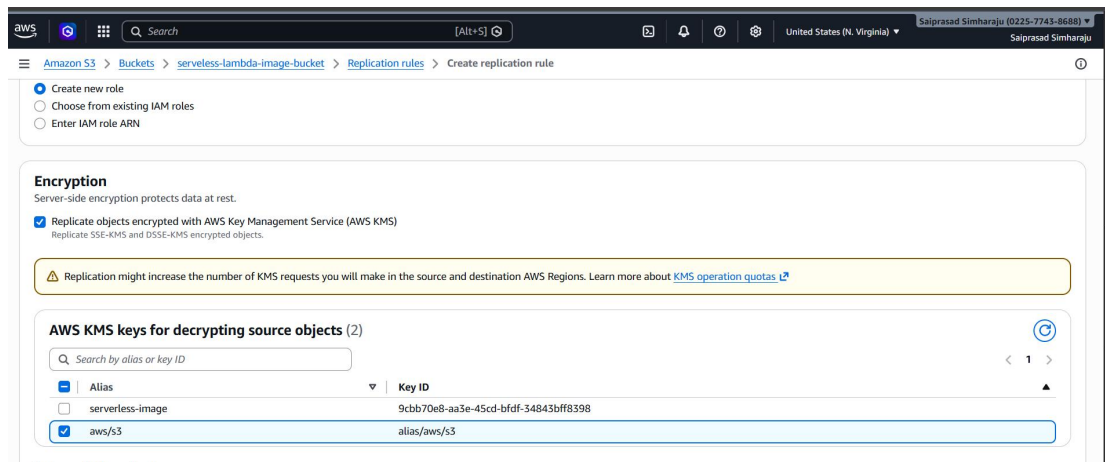
1.3 Create Processed Bucket

- 1. Go to S3 → Create bucket
- 2. Name: `processedimagesbucket`
- 3. Enable Default Encryption → SSEKMS
- 4. Select or create KMS key
- 5. Create bucket

2. CrossRegion Replication (CRR)

- 1. Create destination bucket in euwest1 region
- 2. Enable Versioning on both buckets
- 3. Open `rawuploadsbucket`
- 4. Go to Management → Replication rules
- 5. Create replication rule:
Source prefix: `rawuploads/`
Destination: EU bucket
- 6. Create/attach IAM role
- 7. Save rule

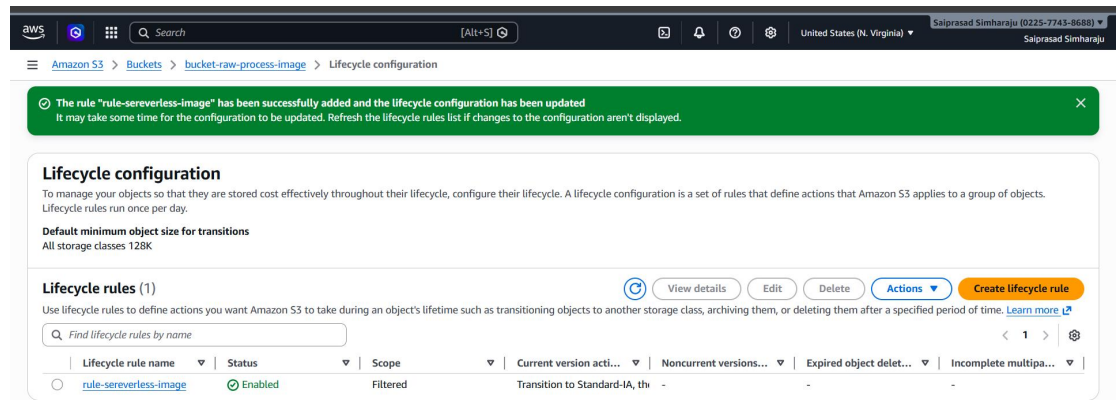
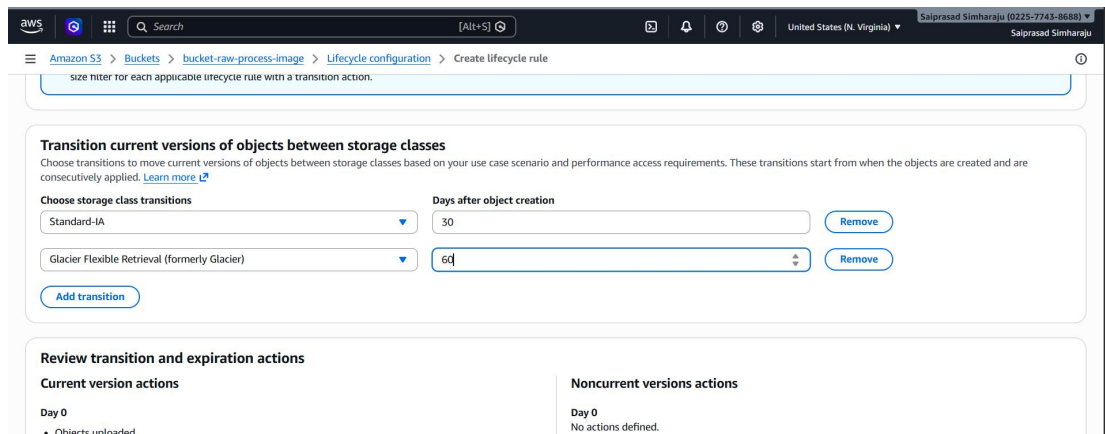
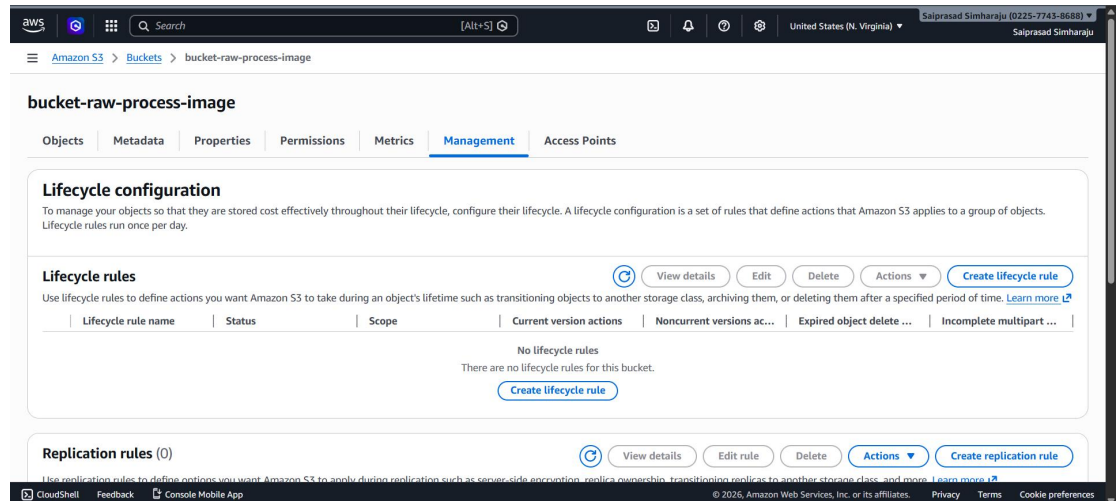
Screenshots:



3. Lifecycle Rules

- 1. Open `rawuploadsbucket`
- 2. Go to Management → Lifecycle rules
- 3. Create rule:
Apply to prefix `rawuploads/`
- 4. Add transitions:
 - Standard → IA after 30 days
 - IA → Glacier after 90 days
 - Glacier → Deep Archive after 365 days
- 5. Save

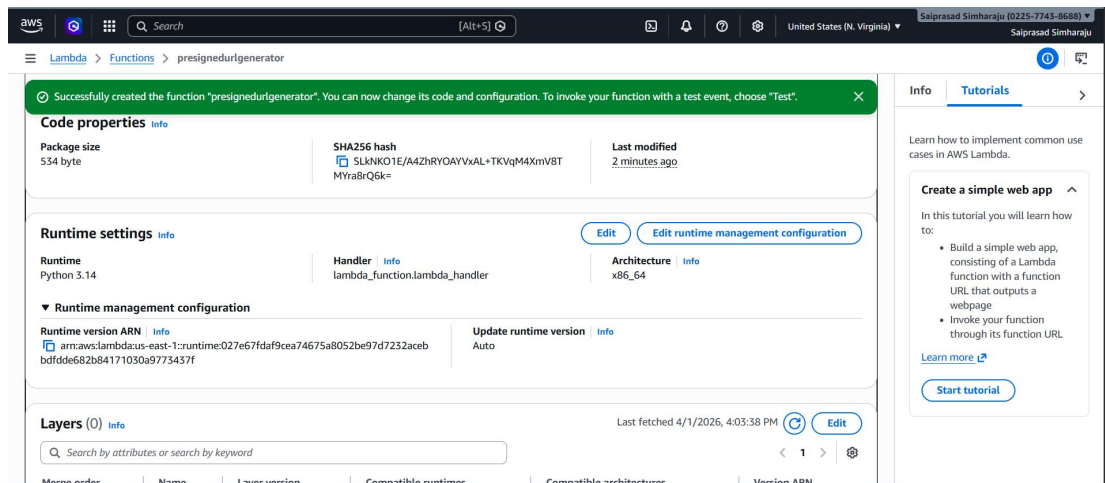
Screenshots:



4. PreSigned URL Setup

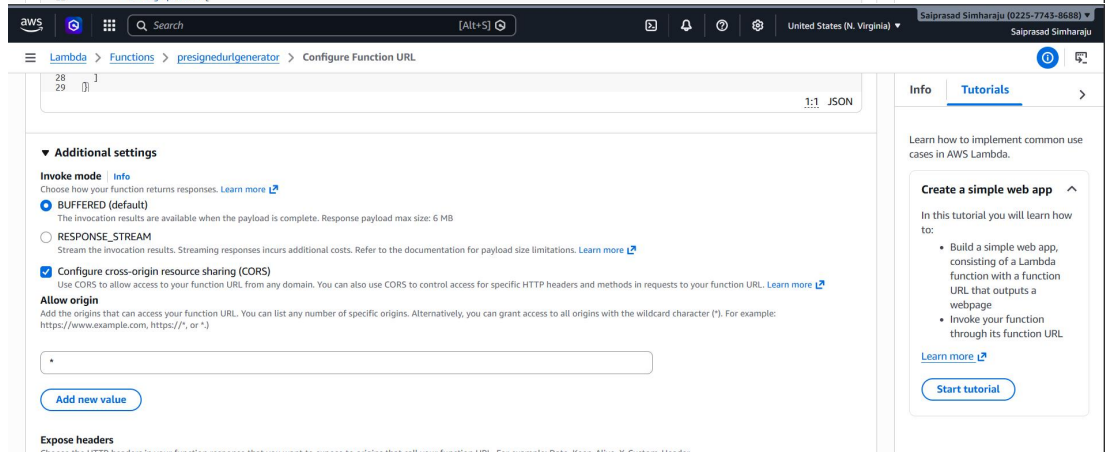
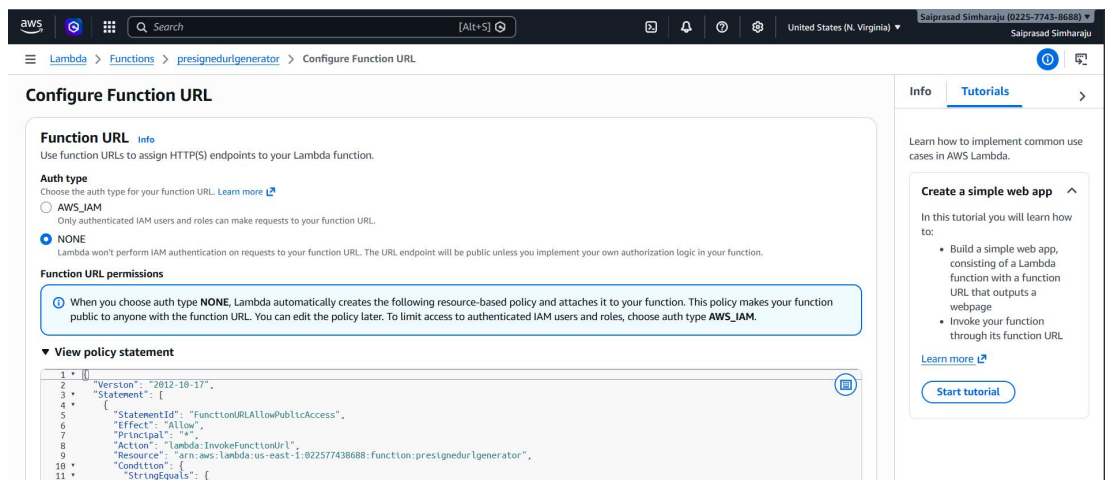
- 1. Use AWS SDK (any environment)
- 2. Generate presigned PUT URL
- 3. Use this URL from browser/client to upload images raw bucket

Screenshots:



Configure function url

Configuration--->function URL-->create url



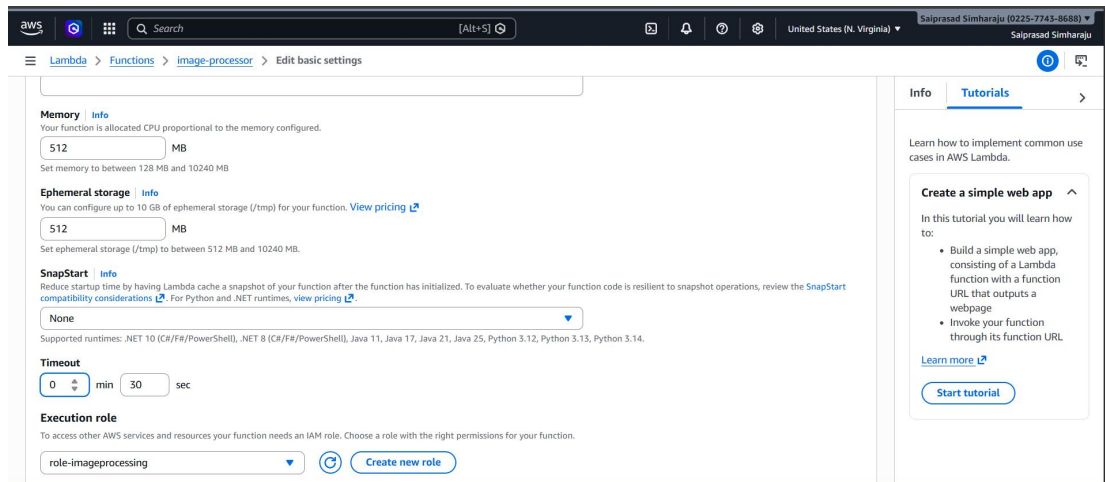
5. Lambda Setup

5.1 Create Function

- 1. Go to AWS Console → Lambda
- 2. Click Create function
- 3. Name: `imageprocessingfunction`
- 4. Runtime: Python 3.11
- 5. Create function

5.2 Configure Settings

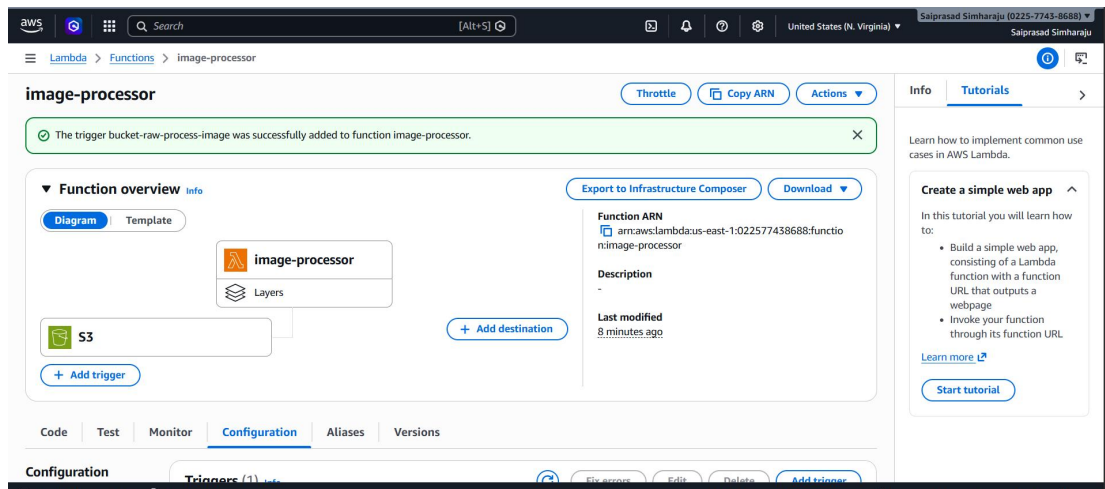
- 1. Go to Configuration → General configuration
- 2. Set:
Memory → 512 MB
Timeout → 30 seconds
- 3. Save



5.3 Add S3 Trigger

- 1. Go to Function overview
- 2. Click Add Trigger
- 3. Select:
Source: S3
Bucket: `rawuploadsbucket`
Event: Object Created
- 4. Add trigger

Screenshots:



5.4 Create Lambda Layer

- 1. Prepare dependency package locally
- 2. Zip the package
- 3. Go to Lambda → Layers
- 4. Click Create layer
- 5. Upload zip
- 6. Select Python 3.11
- 7. Create

Screenshots:

Configure lambda

Environment variables

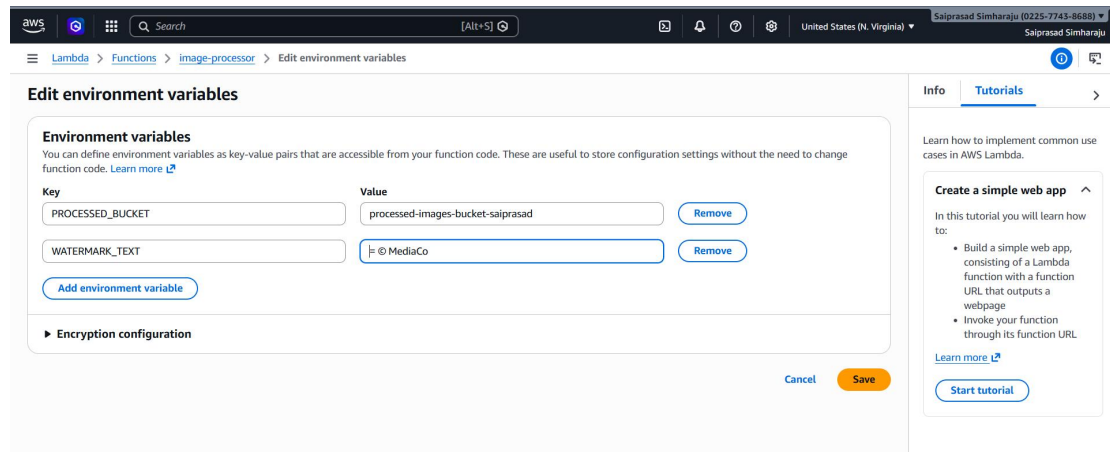


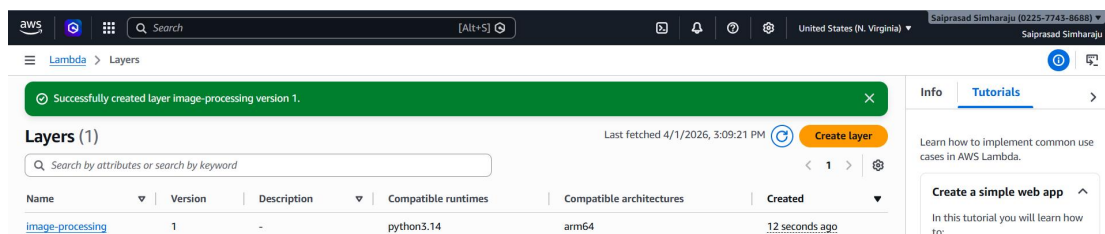
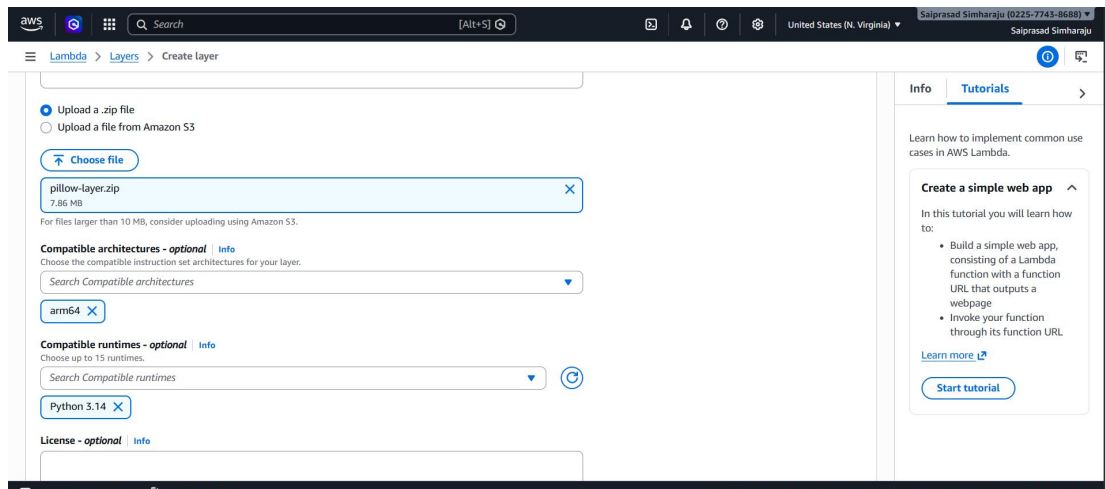
Image processing layer

create layer arn: arn:aws:lambda:us-east-1:022577438688:layer:image-processing:1

edit layer

go to function--->code--->scroll down---->layers---> edit-->add layer-->custom layer-->

Screenshot:



5.5 Attach Layer

- 1. Open Lambda function
- 2. Go to Layers section
- 3. Click Add layer
- 4. Select created layer
- 5. Add

5.6 Add Permissions

- 1. Go to Configuration → Permissions
- 2. Open execution role
- 3. Attach policy:
`AmazonS3FullAccess`

8. Final Validation Steps

- 1. Generate presigned URL
- 2. Upload image via browser

step 1: Generate Pre-Signed URL

Open browser and paste your Lambda Function URL:

Step 2: Copy upload_url

Copy ONLY the value of:

upload_url

Step 3: Go to CloudShell

Check file:

ls

Step 4: Run CURL Command

Paste your URL here:

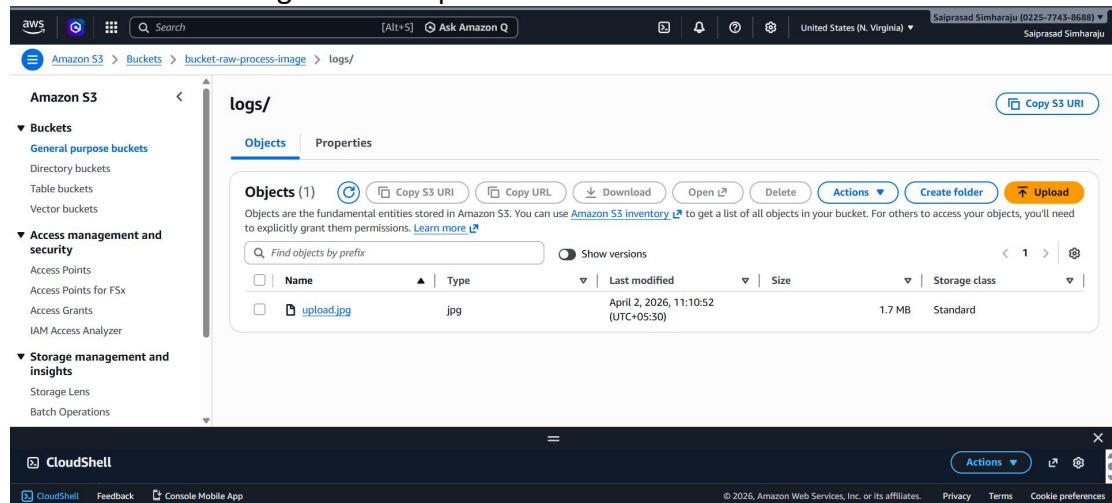
```
curl -X PUT "PASTE_UPLOAD_URL_HERE" \  
-H "Content-Type: image/jpeg" \  
--upload-file test.jpg
```

Step 5: Check Upload

Go to S3:

raw-uploads-bucket-yourname → logs/

- 3. Processed image stored in processed bucket



This covers exactly:

- ✓ S3 setup
- ✓ Presigned uploads
- ✓ Lambda processing
- ✓ Lifecycle rules
- ✓ Crossregion replication